

Project Development Rules & Coding Standards

One-page project standard summary with variable naming rules

1. Project Standard Rules

- Define project scope, milestones, deliverables, and acceptance criteria before development.
- Use task tracking tools and keep requirement, design, API, and deployment documentation updated.
- Create feature branches for new work; keep the main branch stable and release-ready.
- Use meaningful commit messages, review pull requests, and test before deployment.
- Keep folder structure clean; each file, class, and module should have one clear responsibility.
- Use RESTful APIs, consistent JSON responses, proper HTTP status codes, and API documentation.
- Use database migrations, indexes, snake_case table/column names, and input validation.
- Maintain responsive UI, consistent colors/typography, accessibility, and loading performance.
- Protect secrets with environment variables; never commit sensitive information.
- Use authentication, authorization, role-based access control, HTTPS, and dependency updates.

2. Variable Naming Rules

- Use meaningful and readable names that clearly describe the data or action.
- Use camelCase for variables and functions. Example: userName, calculateTotal().
- Boolean names should start with is, has, can, or should. Example: isActive, hasPermission.
- Avoid single-letter names except simple loop counters. Example: i, j.
- Use UPPER_SNAKE_CASE for constants. Example: MAX_RETRY_COUNT, API_BASE_URL.
- Use PascalCase for class names. Example: UserService, PaymentController.
- Use snake_case for database tables and columns. Example: user_profiles, created_at.
- Use lowercase file names with hyphen or underscore. Example: user-service.js, auth_helper.py.
- Component file names should match component names and describe one primary responsibility.

3. Quick Review Checklist

- Readable names? Clean structure? No secrets? Reviewed PR? Tested? Documented?

Condensed from the uploaded 3-page project development rules document.